Product Requirement Document (PRD): Team Task Manager

Version: 1.0
Target Audience: Engineering & Engineering Management (Antigravity)
Status: Approved / Ready for Implementation

1. Executive Summary & Objective

The purpose of this document is to detail the product, functional, technical, and deployment requirements for the **Team Task Manager** web application. This application is a full-stack system designed to enable collaborative project management, task assignment, and delivery lifecycle tracking within organizational teams.
The primary objective is to deliver a live, functional, secure, and highly performant platform featuring robust Role-Based Access Control (RBAC), intuitive dashboards for tracking bottlenecks (such as overdue items), and a scalable RESTful API back-end.

2. User Roles & Permissions (RBAC Matrix)

The application enforces strict server-side and client-side role validation. Users are classified as either **Admin** or **Member** upon account configuration or project setup.

Module / Feature Action	Admin Permissions	Member Permissions
User Authentication & Registration	Allowed (Signup/Login)	Allowed (Signup/Login)
Project Creation & Archival	Full CRUD Permissions	Read-Only View Access
Team Management & Membership Assign	Add/Remove users to projects	No Access
Task Creation & Cross-Assignment	Full CRUD (Assign to anyone)	No Creation (Assigned to self/view)

Module / Feature Action	Admin Permissions	Member Permissions
Task Status Updating (Workflow Lifecycle)	Full Access	Allowed on assigned tasks

3. Functional Requirements & Core Features

3.1 Authentication & Security

- **Secure Signup:** New users can create accounts requiring name, unique email address, and strong password metrics.
- **Secure Login:** Form validation protecting against common injection vulnerabilities, issuing a stateless JSON Web Token (JWT) or secure session cookie.
- **Session Persistence:** Route guards implemented on client applications ensuring non-authenticated requests are redirected back to the login interface.

3.2 Project & Team Management

- Admins must have access to a interface/modal allowing input of Project Name, Description, and Target Deadline.
- Admins can assign organizational Members to the created project workspace. Non-assigned members cannot access or pull metadata regarding that specific project.

3.3 Task Creation, Assignment & Lifecycle Tracking

- **Task Schemas:** Every task object requires Title, Description, Due Date, Status Indicator, Priority Tier, and Assigned User ID.
- **Task Lifecycles:** The core workflow must cleanly process movements across defined states: *To Do*, *In Progress*, and *Done*.
- **Status Mutability:** State updates must fire instantly, utilizing optimized asynchronous network handshakes to minimize UI lockup.

3.4 Analytical Tracking Dashboard

- **Aggregate Metrics:** An upper dashboard display module providing metrics on total tasks, completed counts, pending counts, and critically highlighted overdue counts.
- **Overdue Evaluation Logic:** A dynamic evaluation check comparing dueDate < currentSystemTime where status is NOT Done. These must flags distinct visual warnings to users instantly.

4. Technical Architecture, Schema, & Proper Validations

The platform can utilize either standard relational databases (SQL, e.g., PostgreSQL) or structured document stores (NoSQL, e.g., MongoDB). Data normalization, indexing, and cascade strategies must conform to engineering patterns below.

4.1 Standard Entity Relationships (SQL Sample Design)

`User (id, name, email, password_hash, role)`
- One-to-Many with Project (As Creator/Admin)
- One-to-Many with Task (As Assignee)

`Project (id, title, description, created_by, created_at)`
- Many-to-Many with User (via Project_Members join table)
- One-to-Many with Task (Cascade delete configured)

`Task (id, project_id, assigned_to, title, description, status, due_date)`

4.2 Core REST API Endpoint Specifications

Method	Route	Access Requirements	Validation Constraints
POST	/api/auth/signup	Public	Email verification, min 8 char password
GET	/api/projects	Authenticated (All)	Filters context based on user scope
POST	/api/projects	Admin Only	Title string mandatory, non-empty
PUT	/api/tasks/:id	Admin or Assignee	Status must match

Method	Route	Access Requirements	Validation Constraints
			enum states exclusively

5. Non-Functional & Security Requirements

- **Server Validation Architecture:** Client validation is used only for enhanced UX. The backend server must re-verify all formats, inputs, permissions, lengths, and injection patterns before persisting metadata blocks to database layer storage.
- **Data Isolation protection:** Ensure queries tracking task structures are tightly restricted by project boundaries to completely eliminate horizontal escalation leaks.

6. Deployment Strategy & Selection Milestones

Mandatory Deployment Standard: The engineering artifact must be fully deployed, active, and interactive on **Railway**. Static local code bases without active instances will not meet selection requirements.

6.1 Complete Submission Checklist

1. **Live Production URL:** Accessible interface running seamlessly on production cloud server (Railway).
2. **GitHub Code Repository:** Structured version control containing isolated directories for back-end API code and front-end interface builds.
3. **Comprehensive README File:** Complete step-by-step documentation detailing system environment variables, architecture instructions, structure layouts, and instructions on local reproduction steps.
4. **Demo Presentation Video:** A 2 to 5-minute technical demonstration mapping out authentication, access rules, task assignments, and overdue calculations.